The Depth Flow of Token Representations Is Nonlinear and Does Not Descend Its Own Density

Alexandre Quemy ALEXANDRE@HOTHER.IO
Hother Labs
Centre for Credible AI, Warsaw University of Technology

Editors: Louisa Cornelis, Francisco Acosta, Fatih Dinç, Alison Pouplin, David Klindt, Nina Miolane

Abstract

A token's representation is carried through the network layer by layer. The whole vocabulary carried together forms a flow. We fit this flow's equation of motion as a discrete Langevin model over corpus-mean trajectories of Pythia-160M and Pythia-410M, and score the predicted steps on held-out tokens. Linear maps are often used as cheap surrogates for a layer. The flow they summarize is not linear: a quadratic drift beats the linear map at every transition of both models, and the Kramers–Moyal estimator agrees wherever its neighborhoods stay local. At full width, the step of a mean trajectory is nearly affine, 3% non-affine, while the step of an individual state is not, 17%. We then characterize the flow further. First, we show that it does not descend its own log-density, whatever the diffusion matrix. The drift instead descends a potential that is not the density. Second, the rotational component is not negligible, 4 to 45% of the explainable drift, and it is the antisymmetric part of the linear map. Finally, a token keeps its angular rank and its local-density rank across all thirteen layers, on two corpora and both models.

Keywords: Langevin dynamics, Kramers–Moyal estimator, token representations, transformer depth

1. Introduction

A token's representation is transformed layer by layer via the residual stream, from its embedding row to the state vector used by the model to predict the next token. Across depth, each token traces a trajectory, and the whole vocabulary together forms a flow as depicted in Figure 1.

Writing s_t^ℓ for the residual-stream state at corpus position t after ℓ blocks, and $\mathcal{P}(v)$ for the set of positions whose next token is v, the trajectory of a vocabulary token v is

$$h_v^\ell \;=\; \frac{1}{|\mathcal{P}(v)|} \sum_{t \in \mathcal{P}(v)} s_t^\ell, \qquad \ell = 0, \ldots, L,$$

and the flow is the family of these trajectories over the vocabulary. A trajectory begins at the mean state of the positions that precede v, not at v's own embedding row. We call it the next-token-conditioned mean trajectory of v, and the law we fit is a law for these means.

Concretely, take $v =$ "dog" and a corpus that contains "he walked the dog" and "she fed her dog". Then $\mathcal{P}(\text{dog})$ holds the positions of "the" and "her", the two moments the model is about to produce "dog". Each of those positions carries one vector through thirteen

© 2026 A. Quemy.

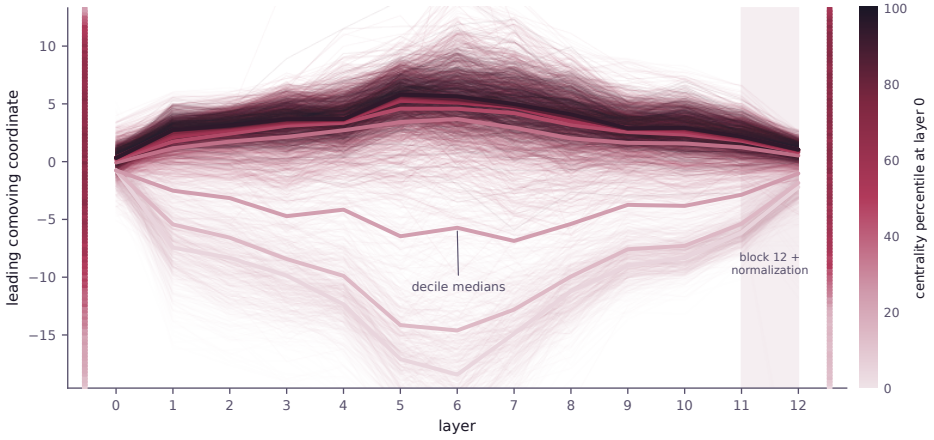


Figure 1: Token trajectories, drawn along the leading comoving coordinate, the axis of largest variation once each layer's cloud is standardized, so a token's height is its position relative to all tokens. Trajectories are colored by the token's centrality percentile at layer 0, its rank in cosine to the ensemble mean direction, bold lines are decile medians, and the side strips give the median color at each height at entry and exit.

states, and averaging them state by state gives the path ${h^0}_{\text{dog}}, \ldots, {h^{12}}_{\text{dog}}$: at layer 0 the mean embedding of "the" and "her", at layer 12 the mean vector from which the model says "dog". Over the full corpus the average runs over every such position.

We are interested in this particular quantity because it is the one vector that the network actually carries end to end, from the embedding to the final state. The average over the corpus is only pragmatic: as the trajectories are context-dependent, they would be most likely unique, such that we would not be able to study them statistically.

In this paper, we study this flow of token representations across depth, and in particular its equation of motion. In other words, how does a model transform a token representation from one layer to the next. We model the flow as a Langevin equation, with a drift and a diffusion term, and we ask whether the drift is linear or nonlinear, and whether it is a gradient flow or not. We use three estimators: with Kramers-–Moyal (KM) expansion (Friedrich et al., 2011), a linear map, and a quadratic map. The linear map is known to transfer across models and scales (Yom Din et al., 2024).

Our contributions are:

- The drift is nonlinear. In a reduced state of 16 components a quadratic drift beats the linear map at every layer of Pythia-160M and Pythia-410M. At full width, the step of a mean trajectory is nearly affine, 3% non-affine, while the step of an individual state is not, 17%.

- The drift is not the gradient of the log-density. We measure the alignment between the drift and the estimated density gradient and find nothing. We show that the motion has a gradient component, but the diffusion and the rotational part are also load-bearing.

- Among four observable quantities of a token in a cloud of tokens (cosine to the mean direction, vector norm, participation ratio and local density), the cosine to the mean direction and the local density are strongly preserved, on two corpora and both models.

2. Related work

A flow of token representations means three different things in the literature. The analytical line treats depth as continuous time and derives the dynamics rather than measuring them: residual networks as discretized flows (Chen et al., 2018), tokens as interacting particles that cluster with depth (Geshkovski et al., 2025), attention as diffusion (Sander et al., 2022). The empirical line follows residual-stream states across layers for individual prompts and measures their geometry (Fernando and Guitchounts, 2025, 2026; Di Sipio et al., 2025). Carson and Reisizadeh (2025) fit a drift-diffusion system, but along generation time rather than depth. Ours is a fourth object: one mean trajectory per vocabulary token, over the corpus, along depth, with the equation fitted from the trajectories and scored on held-out tokens.

The closest prior work to ours is Sarfati et al. (2025). They fit a stochastic equation to layer-wise trajectories and simulate it. Their drift is linear by construction, a rotation and stretch, their noise is isotropic and state-independent, and the gradient reading is never tested. We test all three: the drift is nonlinear (Section 4), the noise is correlated and moves with the state (Section 4), and the gradient reading fails (Section 5). The object also differs: they use a trajectory per prompt, while we use the mean trajectory per vocabulary token.

Yom Din et al. (2024) project hidden states from one layer to another with a linear map, accurately and across model scales. Their map is therefore the baseline any law of motion on depth must be compared to, which we do in Section 4.

Our depth profile is consistent with published geometry. Predictability rising through depth is the iterative-inference literature (Jastrzebski et al., 2018; nostalgebraist, 2020; Geva et al., 2022; Belrose et al., 2023). Early layers hold information in superposition (Lombardo et al., 2026), and depth proceeds in stages (Lad et al., 2025). The embedding table's geometry is organized by frequency (Mu et al., 2018; Gao et al., 2019; Ethayarajh, 2019; Puccetti et al., 2022).

The instruments that we use in this paper are not new either. The Kramers–Moyal expansion is the standard estimator of drift and diffusion from sampled paths (Friedrich et al., 2011), its finite-sampling biases are known (Lade, 2009), and the Langevin–Fokker–Planck correspondence is classical (Risken, 1989). The literature already uses the same equation at least twice, 1) as the continuous limit of stochastic gradient descent (Mandt et al., 2017) and 2) as the backbone of score-based generative models (Song et al., 2021), where the drift is the score by construction.

3. The flow and how we measure it

This section describes the flow, the estimators for the model and the evaluation protocol. The code, the results and the figures of this paper are at https://github.com/hotherio/paper-neurreps-depth-flow.

The trajectories. We use 25,000 WikiText-103 articles that are longer than 400 characters, truncated at 128 tokens for practical reasons. We streamed them through Pythia-160M (Biderman et al., 2023) to collect the thirteen hidden states, and accumulate the means h_v^ℓ defined in Section 1 over the corpus. We then filter out tokens with fewer than five occurrences. The result is a set of 25,268 trajectories of thirteen points in $\mathbb{R}^{768}$.

The reduced state. Because the estimator we will fit uses nearest neighbors, we need to reduce the state dimension. Another reason is that increments in different layers are expressed in different coordinates, and we want to compare them. We fit a principal component basis on a pooled sample of 8,000 tokens across all thirteen layers. To account for the fact that the norm of the residual stream grows with depth, we standardize each layer's cloud to the same size when fitting the basis, dividing each layer's centered states by that layer's radius, and then project the raw states through the single fixed basis. We tried 8, 16, 32, and 64 components, and report the results at 16, which is the working width of Yom Din et al. (2024). At 16, the PCA retains 82.7% of the pooled variance, which is not uniform across layers: 12% of layer 0's own variance, 79% mid-network, 52% at the output.

The model. We model the trajectory of every token as a discrete-time Langevin equation with layer-specific coefficients. Writing $z_v^\ell \in \mathbb{R}^{16}$ for token v's state at depth ℓ (the 768-d mean state projected onto the shared 16-component basis), the model is

$$z_v^{\ell+1} = z_v^\ell + a_\ell(z_v^\ell) + \sqrt{D_\ell(z_v^\ell)}\,\xi_v^\ell, \qquad \xi_v^\ell \sim \mathcal{N}(0, I),$$

with ξ_v^ℓ drawn independently for every token and every transition.

Note that the push a_ℓ and the noise scale D_ℓ do not depend on the token v, since our hypothesis is that the flow is a property of the model: they are fixed functions of position, and a token enters only through its current position z_v^ℓ and its own draw ξ_v^ℓ. D_ℓ is the covariance of what the drift leaves unexplained.

The estimators. Let $\Delta_v = h_v^{\ell+1} - h_v^\ell$ be a token's step. We consider three models to estimate the push a_ℓ and the noise scale D_ℓ from the training tokens:

- *The local reference.* To predict the step at a state x, the Kramers–Moyal expansion (Friedrich et al., 2011) finds the $k = 64$ nearest training states and uses their steps directly, their mean as the push and their covariance as the noise,

$$a(x) = \frac{1}{k}\sum_{u \in \mathcal{N}_k(x)} \Delta_u, \qquad D(x) = \frac{1}{k}\sum_{u \in \mathcal{N}_k(x)} \big(\Delta_u - a(x)\big)\big(\Delta_u - a(x)\big)^\top.$$

- *The affine map.* One matrix and one offset per transition, $h^{\ell+1} = A_\ell h^\ell + b_\ell$, fitted by least squares. This is the baseline of Yom Din et al. (2024). The noise is the covariance of the residuals.

- *The quadratic map.* The same least-squares fit, with the pairwise products of the coordinates added as features. The noise is the covariance of the residuals.

We include nulls for control: fixed Gaussians that never look at the state. Null A fits the mean step of each transition with a per-coordinate variance, null B shares one step distribution across all transitions, and null C gives null A a full covariance.

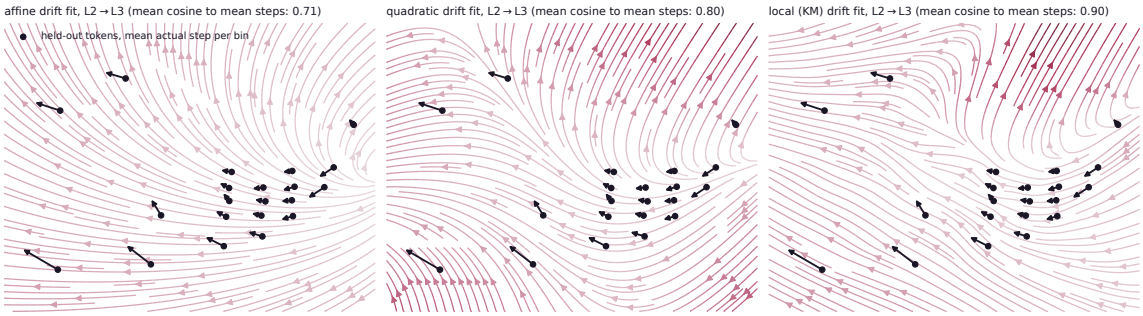


Figure 2: The transition from layer 2 to layer 3 in the leading principal plane of layer 2. In each panel the drift is fitted on these two coordinates only, by the affine map, the quadratic map and the local estimator. Black arrows are the mean held-out step in each bin of the plane.

The protocol. All fits use one 80/20 token split. The target is: given a held-out token's state at layer ℓ, predict its step to layer $\ell + 1$ as a Gaussian, mean from the drift, covariance from the noise model. We use two metrics: 1) R^2 to measure the drift alone, as the fraction of step variance the predicted push explains and 2) mean log-likelihood.

The noise floor. Because we average over occurrences, the trajectories' noise is directly connected to frequency. To measure this noise we split each token's occurrences into two random halves at the final layer and compare the two half-averages, which agree only as well as the full average is reliable. For a token seen five times, the resulting noise is 40% of the typical distance between different tokens' positions. It falls to 21% for tokens seen ten to a hundred times, and to 2% above a thousand. The problem is that the trajectories of rare tokens are noisier by construction which remains one of the limitations of our current protocol.

4. The drift is nonlinear

Both non-linear estimators beat the linear map. At the working width of 16 components, the quadratic map wins all twelve transitions in held-out variance explained (Table 1), and the local law wins eleven of twelve (R^2 of 0.54 to 0.96 against 0.49 to 0.91) and all twelve matched-noise likelihood contests (Table 6 in Appendix C). Figure 2 shows the same bend seen by both where the affine field stays rigid.

Scored against three state-blind nulls, a per-transition Gaussian (A), the same pooled across transitions (B), and the same with full covariance (C), the KM expansion beats A and B on all twelve transitions by 8.4 to 29.1 nats (Table 6).

The diffusion part is important. When both models use the same diagonal noise, the KM expansion beats the linear map twelve of twelve. When the linear map alone gets a full covariance, it wins eight of twelve, and even null C, which has no drift, wins three: the richer noise model wins by itself. When both models get the full covariance, the KM expansion again wins all twelve, by 1.1 to 7.6 nats, with bootstrap intervals excluding zero, at three seeds.

Table 1: Quadratic drift against the affine map, both parametric and both gaining capacity with the state width. Wins are transitions where the quadratic drift explains more held-out increment variance (12 transitions on Pythia-160M, 24 on Pythia-410M). R^2 values are medians over transitions. The last block subsamples the Pythia-160M training set to Pythia-410M's budget, reproducing the 64-component drop and identifying it as an estimation limit.

Model	Train pts	Width	Quad params	Wins	R^2 quad	R^2 affine
Pythia-160M	20,214	8	45	12/12	0.73	0.57
Pythia-160M	20,214	16	153	12/12	0.83	0.69
Pythia-160M	20,214	32	561	12/12	0.84	0.76
Pythia-160M	20,214	64	2145	12/12	0.85	0.81
Pythia-410M	9,549	8	45	24/24	0.76	0.67
Pythia-410M	9,549	16	153	24/24	0.83	0.76
Pythia-410M	9,549	32	561	24/24	0.85	0.78
Pythia-410M	9,549	64	2145	9/24	0.79	0.79
Pythia-160M	9,549	32	561	12/12	0.83	0.76
Pythia-160M	9,549	64	2145	4/12	0.80	0.81

The KM expansion degrades with width: twelve wins at 8 components, five at 32, one at 64. Its fixed 64-point neighborhood grows from 11% to 28% of the cloud radius, and raising k to 256 or 1,024 gives zero wins. This is a failure of the estimator, not a fact about the flow. The quadratic map wins twelve of twelve at all four widths, with medians of 0.73 to 0.85 against the linear map's 0.57 to 0.81 (Table 1), and all twenty-four transitions of Pythia-410M at 8, 16, and 32 components. At 64 on 410M it wins 9 of 24: it needs 2,145 coefficients and has 9,549 trajectories. 160M subsampled to the same budget shows the same drop, 12 of 12 becoming 4 of 12 at 64, unchanged at 32. The largest margin is on the final transition, where the normalization is itself nonlinear. Without it the quadratic still wins 11 of 11, and 23 of 23 on Pythia-410M. We believe that with a larger corpus, and therefore more data points, KM would perform the best at every scale. We leave this to future work.

What the reduction does. The reduced state keeps 16 of 768 coordinates, and a step that is affine in all 768 can still look curved in 16 of them. We checked this with a flow that is affine at full width by construction: a 768-d affine map fitted per transition, applied to the real current states, plus the affine residual of a random other token as noise. In the reduced state it reads almost like the real flow: the quadratic wins 12 of 12 transitions up to 32 components, with 88% of the real margin at 16 components (Table 4 in Appendix C). The same construction on a Gaussian cloud with the real mean and covariance gives the opposite result, 0 of 12: through 16 coordinates an affine flow stays affine when the cloud is Gaussian, and looks curved only because the real cloud is bent. The curvature we see in the reduced state is thus mostly the shape of the cloud, not the map. At full width there is little curvature left to find: an affine map explains 0.57 to 0.91 of the step of the mean trajectories, and letting the map be nonlinear raises this by only +0.008 to +0.033. The average over contexts removes the context-specific part of each block, and the nonlinearity

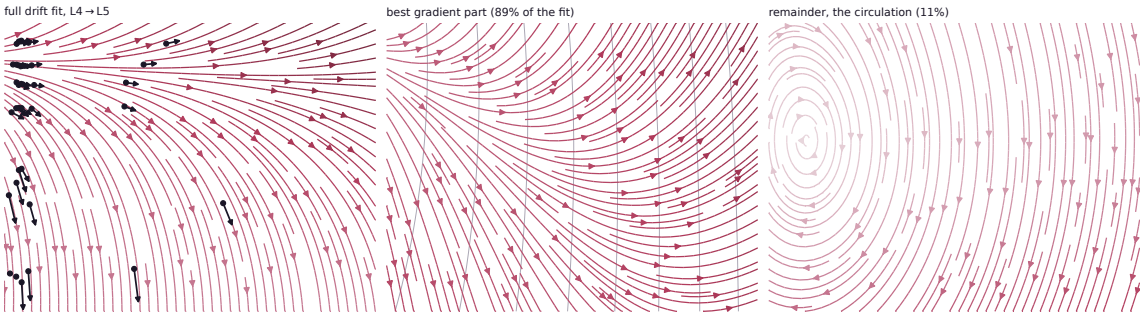


Figure 3: One transition split into descent and circulation. Left: the drift fitted in the leading principal plane of layer 4, with the mean actual held-out step per spatial bin in black. Middle: the best gradient part of that fit, over the contours of its potential. Right: the remainder, a rotation that no potential can produce. Shares are held out. In a single plane most of the circulation is invisible, and the field-level split of Section 5 is the evidence.

is there: on 64,000 individual states the same contest reads a non-affine share of 2% to 25%, median 17%, against 3.4% for the means. On the Pile the quadratic again wins 12 of 12 at every width, and 12 of 12 on 3,024,594 un-averaged positions.

We test whether the law can generate whole trajectories. Each held-out token starts at its real layer-0 state, and we apply the fitted law twelve times in a row, each step adding the predicted push and a draw from the fitted noise, never touching the real intermediate states. The tokens it produces end up too close together: their spread is 0.53 of the real one mid-network, 0.87 by the last raw layer, and a quarter after the final transition. The mid-network figures match the few-layer rollouts of Sarfati et al. (2025). Two artifacts could explain the collapse, and both are ruled out. Iterated local averaging could shrink any cloud by itself, but the same pipeline inflates a stationary control to 2.0, and the composed linear maps inflate to 1.7 rather than shrink. The likely cause is that the fitted noise mixes persistent token identity, the structure that Section 6 will show is preserved, with genuine step-to-step randomness, so the rollout re-draws at every step what a real token keeps. When each simulated token keeps one real token's residual path instead, with the same drift and the same marginal noise size, the spread is 0.93 to 1.19 of the real one on the eleven raw layers, against 0.45 to 0.91 with independent draws, and only the fused final step still halves it (0.47). A law with a per-token component held fixed along the trajectory is left to future work.

5. The drift is not the gradient of the log-density

We write the drift as a gradient of the log-density:

$$a(x) \;=\; D(x)\,\nabla \log p(x),$$

If this reading is true, it would mean that a layer is diffusing particles towards dense regions. This is a different view from the energy-descent literature (Zimin et al., 2026) where a layer

is an optimizer: attention descends an interaction energy between the positions, the MLP descends a potential. Both could be true since the energy-based approach is about what the architecture computes given a context, while our approach is concerned with the log-density over the whole vocabulary.

We estimate the density gradient direction at every token from its $k = 48$ nearest neighbors and measure its mean cosine with the token's actual increment. The reading fails at every transition. Against a shuffle null that pairs each increment with another token's gradient, the mean alignment gap is -0.008, with per-transition gaps between -0.116 and $+0.053$. The negative results are valid for all state dimensions (8 to 64 components) and for both Pythias' parameter counts (Table 2 in Appendix A). To validate the negative result, we create a null and calibrated matched process (Appendix A). The result does not depend on the diffusion matrix. If $a = D\,\nabla \log p$ for some positive-definite D, then $a \cdot \nabla \log p > 0$ at every state, whatever D is. On controls with an isotropic, an anisotropic and a state-dependent D, the product is positive for every token. On the real flow it is positive for 48% of the tokens, and for 49% when each increment is paired with another token's gradient (Appendix B). The null holds on the Pile too (mean gap -0.006, largest +0.094), and on the un-averaged positions (mean gap 0.05 against 0.90 on the control).

The negative result for the log-density does not mean the drift is not the gradient of some potential. The best cubic potential fitted to the centered drift recovers 55 to 96% of what the unconstrained quadratic field explains, leaving 4 to 45% of rotational part (Figure 3). We controlled where this rotation comes from with the two surrogate flows of Table 4. The flow that keeps the real 768-d linear map and drops the nonlinearity rotates as much as the real flow, 3% to 42%. The flow that keeps only the symmetric part of that map, which cannot rotate at full width, still reads 1% to 23% in the reduced state. The rotation is thus in the linear map of the network, and the reduction adds at most 23% on top. The rotational part itself is one minus that ratio, $1 - R^2_{\text{grad}}/R^2_{\text{quad}}$, with both R^2 held out. Its value moves with the ridge of the potential fit, 4% at the last transition becoming 19% at a hundred times the ridge, but the order of the transitions does not. At linear order, the symmetric share of the fitted map is 0.53 to 0.84 against shuffle nulls of 0.50 to 0.54, largest at the final layer, as in Fernando and Guitchounts (2026).

The consequence is that no single landscape summarizes the flow: the drift mostly descends a potential that is not the log-density, the remainder circulates, and the correlated noise carries the rest of the law. This is coherent with what we observe in Section 6 and in Figure 1, where the rank of the cosine to the mean direction and the rank of the local density are preserved by the flow while it moves everything.

6. What the flow preserves

For each layer, we measure four observables: the cosine to the ensemble mean direction, the vector norm, the participation ratio $(\sum_i h_i^2)^2 / \sum_i h_i^4$, and the local density, minus the distance to the tenth nearest token at that layer.

We calculate the intraclass correlation of the token population,

$$\mathrm{ICC}(f) \;=\; \frac{\mathrm{Var}_v\left[\mathbb{E}_\ell\, \zeta_v^\ell(f)\right]}{\mathrm{Var}_v\left[\mathbb{E}_\ell\, \zeta_v^\ell(f)\right] \;+\; \mathbb{E}_v\left[\,\mathrm{Var}_\ell\; \zeta_v^\ell(f)\right]},$$

where $\zeta_v^\ell(f)$ is the per-layer z-score of the observable f for token v, distinct from the reduced state z_v^ℓ of Section 3.

The cosine is preserved with a Spearman rank correlation of 0.76 between entry and exit, and an ICC of 0.75 on WikiText, 0.67 on the Pile and 0.77 on Pythia-410M. The local density is preserved at least as much, with an ICC of 0.79 to 0.87 on the same three sets. On Pythia-160M, the norm is shuffled -0.17 while participation ratio is partly reversed at -0.59. The shuffle happens during the layers, as opposed to the participation ratio that is being reversed only at the last layer. On Pythia-410M both are positively preserved ($+0.47$ and $+0.59$). Therefore, further models and model sizes are needed to confirm these results.

7. Conclusion

We fitted an equation of motion to the depth flow of token representations: 25,268 corpus-mean trajectories over thirteen states of Pythia-160M and Pythia-410M, a discrete Langevin model, three estimators, all scored on held-out tokens.

We showed three things. First, the drift is nonlinear: the quadratic drift beats the linear map on Pythia-160M and Pythia-410M in a reduced state, and at full width individual states keep a nonlinear part of 17% of their step. Only the mean trajectories are nearly affine at full width. Second, the drift does not descend its own log-density: the alignment reads nothing at every width and on both models, against a calibrated instrument, while the best cubic potential shows the drift mostly descends some other landscape and 4 to 45% of it circulates. Finally, the flow preserves a token's angular rank and its local-density rank across all thirteen layers, and on Pythia-160M its norm rank is shuffled and its concentration rank is reversed by the last block.

However, our findings have several limitations. First, we studied only one family of model (Pythia) and at small scale. Second, the corpus is likely to be too small for the Kramers–Moyal expansion to properly shine. Third, the law is a regression of the increments: its noise term is what the drift leaves unexplained, not a diffusion we have established. Finally, we do not know which part of the block, attention or the MLP, carries the nonlinear remainder. Therefore, the next immediate steps are to replicate the study with different families of models, possibly at larger scale.

Another limitation is against the literature. The fitted stochastic differential equation of Sarfati et al. (2025) effectively serves as a surrogate of the trajectories, while ours does not work yet as such. Section 4 shows that the fitted noise contains token identity, and that keeping it fixed along the rollout restores the spread. Solving this problem would open the door to the main practical application of such motion law, namely a better surrogate. Another axis around the same question is the study of un-averaged trajectories. Finally, it remains to explore the identity of the potential that the drift does descend. We showed the component exist, but we have not identified it yet.

To conclude, token representations do not slide down a landscape into their predictions: they are pushed by a field that keeps their angular order while it moves the whole vocabulary.

References

Nora Belrose, Zach Furman, Logan Smith, Danny Halawi, Igor Ostrovsky, Lev McKinney, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. arXiv:2303.08112, 2023.

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2304.01373.

Jack David Carson and Amir Reisizadeh. A statistical physics of language model reasoning. arXiv:2506.04374, 2025.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Riccardo Di Sipio, Jairo Diaz-Rodriguez, and Luis Serrano. The curved spacetime of transformer architectures. arXiv:2511.03060, 2025.

Kawin Ethayarajh. How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings. In *Proceedings of EMNLP-IJCNLP*, 2019.

Jesseba Fernando and Grigori Guitchounts. Transformer dynamics: A neuroscientific approach to interpretability of large language models. arXiv:2502.12131, 2025.

Jesseba Fernando and Grigori Guitchounts. Dynamics of the transformer residual stream: Coupling spectral geometry to network topology. arXiv:2605.14258, 2026.

Rudolf Friedrich, Joachim Peinke, Muhammad Sahimi, and M. Reza Rahimi Tabar. Approaching complexity by stochastic methods: From biological systems to turbulence. *Physics Reports*, 506(5):87–162, 2011.

Jun Gao, Di He, Xu Tan, Tao Qin, Liwei Wang, and Tie-Yan Liu. Representation degeneration problem in training natural language generation models. In *International Conference on Learning Representations (ICLR)*, 2019.

Borjan Geshkovski, Cyril Letrouit, Yury Polyanskiy, and Philippe Rigollet. A mathematical perspective on transformers. *Bulletin of the American Mathematical Society*, 62(3):427–479, 2025. arXiv:2312.10794.

Mor Geva, Avi Caciularu, Kevin Ro Wang, and Yoav Goldberg. Transformer feed-forward layers build predictions by promoting concepts in the vocabulary space. In *Proceedings of EMNLP*, 2022.

Stanislaw Jastrzebski, Devansh Arpit, Nicolas Ballas, Vikas Verma, Tong Che, and Yoshua Bengio. Residual connections encourage iterative inference. In *International Conference on Learning Representations (ICLR)*, 2018.

Vedang Lad, Jin Hwa Lee, Wes Gurnee, and Max Tegmark. The remarkable robustness of LLMs: Stages of inference? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2406.19384.

Steven J. Lade. Finite sampling interval effects in Kramers–Moyal analysis. *Physics Letters A*, 373(41):3705–3709, 2009.

Gianfranco Lombardo, Giuseppe Trimigno, and Stefano Cagnoni. A geometric perspective on next-token prediction in large language models: Three emerging phases. arXiv:2605.09011, 2026.

Stephan Mandt, Matthew D. Hoffman, and David M. Blei. Stochastic gradient descent as approximate bayesian inference. *Journal of Machine Learning Research*, 18(134):1–35, 2017.

Jiaqi Mu, Suma Bhat, and Pramod Viswanath. All-but-the-top: Simple and effective post-processing for word representations. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1702.01417.

nostalgebraist. Interpreting GPT: the logit lens. LessWrong, technical report, 2020.

Giovanni Puccetti, Anna Rogers, Aleksandr Drozd, and Felice Dell'Orletta. Outlier dimensions that disrupt transformers are driven by frequency. In *Findings of EMNLP*, 2022.

Hannes Risken. *The Fokker-Planck Equation: Methods of Solution and Applications*. Springer, 2nd edition, 1989.

Michael E. Sander, Pierre Ablin, Mathieu Blondel, and Gabriel Peyré. Sinkformers: Transformers with doubly stochastic attention. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2022. arXiv:2110.11773.

Raphaël Sarfati, Toni J. B. Liu, Nicolas Boullé, and Christopher J. Earls. Lines of thought in large language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.01545.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

Alexander Yom Din, Taelin Karidi, Leshem Choshen, and Mor Geva. Jump to conclusions: Short-cutting transformers with linear transformations. In *Proceedings of LREC-COLING 2024*, pages 9615–9625, 2024. arXiv:2303.09435.

Aleksandr Zimin, Yury Polyanskiy, and Philippe Rigollet. Yuriiformer: A suite of nesterov-accelerated transformers. arXiv:2601.23236, 2026.

Appendix A. The gradient test and its calibration

A null result needs a floor. A near-zero alignment could mean the flow is not a gradient flow, or it could mean the instrument cannot see one. The control samples 25,268 points from a stationary Gaussian with per-coordinate scales log-spaced between 1 and 4, sets $a = D\,\nabla \log p$ with $D = I$, and takes one Euler step per point at a chosen drift-to-noise ratio, matched to the real flow in sample size, dimension, and k. The estimated gradient direction agrees with the analytic one at cosine 0.914. Across drift-to-noise ratios from 0.9 to 106 it reads alignments of 0.61 to 0.91 against shuffle nulls within 0.004 of zero, and in energy terms places 43 to 81% of drift energy along the estimated gradient, against 6 to 29% on the real flow. A gradient flow would have lit this instrument up.

Table 2: The gradient reading and its floor. For each state width the same estimator is run on the real flow and on a matched Ornstein–Uhlenbeck process for which $a = D\,\nabla \log p$ holds by construction. Real values are means over the twelve transitions of the alignment gap over a shuffle null.

Width	16	32	64	128
Real flow, mean gap	−0.008	−0.009	−0.010	−0.004
Real flow, largest gap	+0.053	+0.056	+0.053	+0.050
Ornstein–Uhlenbeck control	0.898	0.854	0.819	0.791

And the reading survives the common translation. Most of each increment is a shared translation of the whole cloud, 76% of increment energy in median, and a translation carries no alignment of its own, so a residual gradient component could hide beneath it: injecting that translation share into the control drops its reading from 0.90 to 0.42. Removing each transition's mean step restores the control to 0.90, and the real flow still reads nothing on the token-specific residuals, a mean gap of −0.02 with a maximum of +0.07.

Appendix B. The reading for any diffusion matrix

Table 3 gives the sign test of Section 5 at four widths. The controls are stationary Gaussian clouds with per-coordinate scales log-spaced between 1 and 4, one Euler step at drift-to-noise ratios 2 and 5, with the drift $D\,\nabla \log p$ for three choices of D: the identity; a constant matrix with eigenvalues log-spaced from 1 to 16 in a basis rotated against the cloud's axes; and $D(x) = (1+|x|^2/c)\,D_0$ with the divergence term added to the drift so that the Gaussian stays stationary. The drift of a held-out token is the local mean step of its 64 nearest training tokens and the score direction the unit mean shift of its 48 nearest tokens.

Table 3: The sign test at four widths: the fraction of held-out tokens whose drift has a positive inner product with the estimated score. Controls is the smallest value over the six controls of Appendix B, real and shuffled are means over transitions, then the per-transition range, the same with the common translation removed, and the cosine gap once the score is multiplied by the fitted residual covariance.

width	controls	real	shuffled	per transition	translation removed	cos. gap with $\hat{D}$
16	1.00	0.479	0.490	0.37–0.57	0.32–0.59	-0.012 / +0.055
32	1.00	0.483	0.494	0.41–0.59	0.30–0.61	-0.013 / +0.041
64	1.00	0.483	0.495	0.40–0.59	0.28–0.60	-0.010 / +0.039
128	1.00	0.489	0.500	0.39–0.59	0.29–0.60	-0.010 / +0.041

Table 4: Quadratic against affine map on the real flow and on four surrogates whose full-width step is known. Wins are transitions where the quadratic map explains more held-out increment variance, margins are medians over transitions, and the last column is the range of the circulation share at 16 components. Cross-fitted: every training token predicted by a map fitted on the other half.

	16 components		32		64		circulation (16)
flow	wins	margin	wins	margin	wins	margin	
real flow	12/12	+0.083	12/12	+0.068	12/12	+0.042	0.04–0.45
affine at full width	12/12	+0.074	12/12	+0.061	10/12	+0.018	0.03–0.42
affine, cross-fitted	12/12	+0.074	12/12	+0.058	9/12	+0.015	–
affine, Gaussian cloud	0/12	-0.002	0/12	-0.006	0/12	-0.021	–
gradient at full width	12/12	+0.014	12/12	+0.009	6/12	+0.001	0.01–0.23

Table 5: The non-affine part of the step per state width, on the mean trajectories and on 64,000 individual states. Affine R^2 is held out; gains are what the network adds to the affine map on the null and on the real flow; recovery is the fraction of a planted 10% non-affine term the network finds; the last column is the real gain over the null read through the recovery. Ranges and medians over the twelve transitions of Pythia-160M.

width	affine R^2	gain on null	gain on real	real − null	recovery	non-affine
mean trajectories						
raw 768	0.58–0.91	-0.002 to -0.000	+0.008 to +0.033	+0.019	0.55	3.4%
256	0.61–0.93	+0.004 to +0.033	+0.020 to +0.060	+0.024	0.51	4.5%
64	0.58–0.93	+0.022 to +0.124	+0.030 to +0.150	+0.023	0.60	3.8%
16	0.49–0.91	+0.036 to +0.304	+0.040 to +0.327	+0.020	0.67	2.1%
individual states						
raw 768	0.45–0.92	-0.001 to -0.000	+0.017 to +0.151	+0.085	0.62	16.7%
64	0.31–0.88	+0.040 to +0.161	+0.061 to +0.453	+0.089	0.47	19.6%
16	0.20–0.86	+0.050 to +0.343	+0.062 to +0.588	+0.051	0.24	19.9%

Appendix C. The likelihood contests and the projection control

Table 6 is the per-transition likelihood table of Section 4, Table 4 the projection control of the same section, and Table 5 its full-width contest on the mean trajectories and on individual states.

Table 6: Held-out log-likelihood per increment on the twelve transitions of Pythia-160M, in nats, higher is better, best per row in bold. KM is the fitted local law with per-coordinate diffusion, KM-full its variant with a local full covariance ($k = 64$, off-diagonals shrunk by 0.1). Null A is the per-transition Gaussian, null B the same pooled across transitions, null C the full-covariance per-transition Gaussian. Affine is the least-squares map with per-coordinate or full residual covariance. The R^2 columns give held-out increment variance explained by each drift.

Transition	KM	KM-full	Null A	Null B	Null C	Affine	Affine-full	R^2 KM	R^2 aff.
L0→1	5.61	**8.59**	-8.93	-44.55	-2.04	-0.97	2.38	0.83	0.73
L1→2	14.94	**16.82**	4.77	-43.95	10.57	11.86	14.72	0.55	0.49
L2→3	6.96	**11.29**	-4.83	-44.04	3.55	4.62	8.53	0.72	0.66
L3→4	8.00	**9.91**	-5.66	-45.16	2.84	4.46	7.73	0.78	0.58
L4→5	11.90	**14.04**	-1.74	-43.97	9.01	9.48	12.83	0.84	0.81
L5→6	5.50	**12.49**	-2.91	-43.97	7.23	2.73	10.81	0.54	0.50
L6→7	4.46	**11.93**	-4.74	-44.00	5.98	2.25	10.28	0.65	0.59
L7→8	-0.38	**6.80**	-12.22	-44.63	0.54	-1.80	5.57	0.80	0.78
L8→9	-4.97	**2.75**	-17.06	-46.92	-4.73	-5.51	1.64	0.77	0.79
L9→10	-5.95	**0.31**	-16.58	-45.02	-8.03	-8.06	-1.96	0.76	0.72
L10→11	-6.89	**-0.97**	-25.88	-48.63	-10.80	-9.01	-3.49	0.92	0.91
L11→12	-14.06	**-12.30**	-43.15	-117.53	-28.82	-32.16	-19.91	0.96	0.66